AI – POWERED INTELLIGENT MEDICAL SYSTEM

Bonafide record of work done by

Anbuchandiran K (23Z209)

Priyadharshini K (23Z250)

Rechivarthini S K (23Z252)

Saumiyaa Sri V L (23Z261)

Therdhana J P (23Z273)

23Z611 – INNOVATION PRACTICES LAB

GUIDE : Ms. Naga Sudha C M

Dissertation submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

BRANCH: COMPUTER SCIENCE AND ENGINEERING



APRIL 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PSG COLLEGE OF TECHNOLOGY

(Autonomous & an ISO 90001 certified Institution)

COIMBATORE – 641 004

PSG COLLEGE OF TECHNOLOGY

(Autonomous & an ISO 90001 certified Institution)

COIMBATORE – 641 004

Bonafide record of work done by

Anbuchandiran K (23Z209)

Priyadharshini K (23Z250)

Rechivarthini S K (23Z252)

Saumiyaa Sri V L (23Z261)

Therdhana J P (23Z273)

Dissertation submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

BRANCH: COMPUTER SCIENCE AND ENGINEERING

of Anna University

APRIL 2026

Ms. Naga Sudha C M	**Dr. Sudha SadhasivamG**
Faculty Guide	**Head of the Department**

Certified that the candidate was examined in the viva-voce examination held on …….

……	…….
(Internal Examiner)	**(External Examiner)**

CONTENTS

TABLE OF CONTENTS

Certificate from the Industry……5
Originality Certificate……6
Acknowledgement……7
Synopsis……8

1. Introduction……9
   1.1 Problem Statement……9
   1.2 Objectives……10
   1.3 Role of AI and Chat System……11
   1.4 Approach……11

2. Literature Survey……12
   2.1 OpenAI Integration……12
   2.2 Significance and Impact……12
   2.3 OpenAI Components……13
      2.3.1 Language Model……13
      2.3.2 Reinforcement Learning Algorithms……14
      2.3.3 Safety and Ethical AI Research……14
      2.3.4 Open Source Tools and Frameworks……14
      2.3.5 APIs and Services……15
      2.3.6 Research Papers and Publications……15

3. System Requirements……16
   3.1 Functional Requirements……16
   3.2 Non-Functional Requirements……17
   3.3 Hardware Requirements……17
      3.3.1 Windows Deployment……17
         3.3.1.1 Minimum Specifications……18
         3.3.1.2 Recommended Specifications……18
      3.3.2 macOS Deployment……18
         3.3.2.1 Minimum Specifications……18
         3.3.2.2 Recommended Specifications……19
   3.4 Software Requirements……19
   3.5 Security Considerations……20

4. System Design and Implementation……..21
4.1 Architecture of the Proposed System 21
4.2 Dataset Description 22
4.3 Model Used 22
4.4 Core Functionalities..... 23
4.5 Implementation..... 23
4.5.1 Variants of the OpenAI Model and Preparing GPT-4 24
4.5.2 Text Preprocessing..... 24
4.5.3 Text Extraction and Preprocessing for Vectorization……...24
4.5.4 Generating Vector Representations with OpenAI Embeddings 24
4.5.5 Building and Managing the FAISS Vector Database 25
4.5.6 Integration with OpenAI Cloud and FAISS CPU 25
4.5.7 Custom Prompt Engineering for Enhanced Model Performance….25
4.5.8 Implementing Query Processing and Response Generation 26
4.5.9 System Monitoring and Function Tracking for Optimization 26

5. Performance Metrics 34

6. Conclusion and Future Scope.....35
6.1 Summary 35
6.2 Future Scope.....35

References 37
Appendix – Source Code…….42
Appendix – Output.....43
Plagiarism Report.....44

CERTIFICATE

This is to certify that the project titled "**AI powered Intelligent Medical System"** was completed by the students studying III – BE – CSE- G1 at PSG College of Technology, Coimbatore, under the guidance of **Ms. Naga Sudha C M** and the duration of the project is from 08.12.2025 to 08.04.2026. This project was carried out as part of the "CubeAISolutions Tech Pvt Ltd "executed by Ms. Swathi Senthil.

The students involved in the project are given below:

1. 23Z209 – Anbuchandiran K
2. 23Z250 – Priyadharshini K
3. 23Z252 – Rechivarthini S K
4. 23Z261 – Saumiyaa Sri V L
5. 23Z273 – Therdhana J P

Date :

Place : Coimbatore

Ms. Naga Sudha C M

Assistant professor

Department of Computer Science & Engineering

PSG College of Technology

Coimbatore- 641004.

COUNTERSIGNED

Dr.G.Sudha Sadasiva

Professor & Head

Department of Computer Science & Engineering

PSG College of Technology

Coimbatore-641004.

ACKNOWLEDGEMENT

We would like to extend our sincere thanks to **Dr. K. Prakasan**, **Principal**, for providing us with this opportunity to develop and complete our project in our field of study.

We express our sincere thanks to **Dr. Sudha Sadasivam G, Head of the Department & Professor**, Department of Computer Science and Engineering, for encouraging and allowing us to present the project in our department premises as partial fulfillment of the requirements for the award of the B.E. degree.

We extend our sincere appreciation to **Dr. Arul Anand N, Programme Coordinator**, Faculty Guide & Professor, Department of Computer Science and Engineering, PSG College of Technology, Coimbatore. His unwavering support and encouragement have been instrumental in our continuous skill enhancement throughout this project. His invaluable suggestions, encouragement, whole-hearted cooperation, constructive criticism, and guidance have significantly contributed to the success of this project.

We extend our heartfelt appreciation to **Ms. Naga Sudha C M, Faculty Mentor,** Assistant Professor (Sl. Gr), Department of Computer Science and Engineering, PSG College of Technology, Coimbatore, for her invaluable support, unwavering encouragement, insightful feedback, and dedicated mentorship throughout this project journey. Her contributions have been instrumental in shaping our work and fostering our growth. We are truly grateful for her steadfast guidance and whole-hearted cooperation, which have greatly enriched our learning experience and contributed to the successful completion of this project.

SYNOPSIS

Title: MedX – Real-Time AI Medical Assistant

The MedX project is a Real-Time AI-based Medical Assistant designed to enhance healthcare workflow efficiency and improve patient safety. In many hospital environments, critical patient data such as prescriptions, case sheets, and nursing notes are often maintained in handwritten or unstructured formats. This makes it difficult to access, verify, and analyze information quickly, leading to delays in decision-making and an increased risk of medical errors.

The proposed system addresses these challenges by converting medical data into structured digital format and enabling intelligent analysis using Artificial Intelligence techniques. The system uses Natural Language Processing (NLP) to understand medical text and extract relevant information such as drug names, dosage details, and patient-related data.

To improve information retrieval, the system generates vector representations (embeddings) of the processed data and stores them in a FAISS-based vector database. This allows efficient similarity search and retrieval of relevant medical information. When a user interacts with the system through a chat-based interface, the query is processed, relevant data is retrieved from the database, and an appropriate response is generated using an AI language model.

The system is also capable of identifying potential drug-drug interactions and providing alerts, thereby helping healthcare professionals make safer and more informed decisions. The chat interface enables users to access patient information and verify prescriptions in real time.

Overall, the MedX system aims to reduce manual effort, improve accessibility to medical data, and assist healthcare professionals by providing accurate, real-time AI-driven support. Future enhancements may include advanced features such as voice-based interaction and integration with hospital management systems.

1. INTRODUCTION

Healthcare systems generate a large amount of patient data every day in the form of prescriptions, case sheets, nursing notes, and medical reports. Much of this data exists in handwritten or unstructured formats, making it difficult to store, retrieve, and analyze efficiently. In busy hospital environments, doctors and nurses must make quick decisions, and any delay in accessing patient information or verifying prescriptions can affect patient safety.

With the advancement of Artificial Intelligence (AI), it is now possible to build intelligent systems that can understand medical text, digitize handwritten documents, analyze prescriptions, and assist healthcare professionals in real-time decision-making. AI technologies such as Natural Language Processing (NLP), Machine Learning, Deep Learning, and Reinforcement Learning can be used to develop systems that improve clinical workflow efficiency and reduce medical errors.

The MedX project focuses on developing a **Real-Time AI Medical Assistant** that digitizes medical records, validates prescriptions, predicts drug interactions, and provides an interactive chat-based system for healthcare professionals. The system integrates AI models, vector databases, and retrieval-based techniques to provide accurate and fast responses to medical queries.

The goal of this project is not to replace doctors but to **assist healthcare professionals** by providing intelligent insights, reducing manual work, and improving patient safety through automated prescription analysis and decision support systems.

1.1 Problem Statement

In modern healthcare environments, critical patient data such as prescriptions, case sheets, and nursing notes are often written by hand or stored in unstructured formats. These records are difficult to digitize, verify, and access instantly when required. This lack of digital accessibility creates delays in decision-making and increases the risk of medical errors during real-time patient care.

One of the major challenges in hospitals is **interpreting handwritten prescriptions**. Different doctors have different handwriting styles, and sometimes prescriptions may be unclear or difficult to read. This can lead to incorrect drug dispensing or dosage errors. Another major issue is **drug-drug interactions**, where multiple drugs prescribed to a patient may interact negatively and cause harmful side effects. Detecting these interactions manually for every prescription is time-consuming and not always reliable.

In addition, patient medical history is often stored in multiple formats such as paper records, PDFs, or different hospital databases. This makes it difficult for doctors to quickly retrieve relevant patient information while making treatment decisions.

There is also a growing need for intelligent systems that can understand unstructured medical data, validate prescriptions, and assist healthcare professionals by providing real-time recommendations and alerts. Without such systems, hospitals rely heavily on manual processes, which increases workload and the chances of human error.

Therefore, there is a need to develop a **Real-Time AI Medical Assistant** that can digitize handwritten medical records, analyze prescriptions, detect potential drug interactions, and provide instant responses to medical queries through an intelligent chat-based interface.

1.2 Objectives

The main objective of the MedX project is to develop an AI-based system that assists healthcare professionals in managing medical data, validating prescriptions, and improving clinical workflow efficiency.

The specific objectives of the project are as follows:

1. **Digitization of Medical Records**
   To convert handwritten prescriptions, case sheets, and nursing notes into structured digital text using text extraction and preprocessing techniques.
2. **Prescription Validation**
   To analyze prescriptions and identify potential drug-drug interactions using AI models and retrieval-based techniques.
3. **Medical Data Understanding**
   To use Natural Language Processing (NLP) to understand unstructured medical text and extract important information such as drug names, dosage, and patient details.
4. **Embedding and Vector Search**
   To convert medical text into vector representations using embeddings and store them in a vector database for efficient retrieval of relevant medical information.
5. **Chat-Based AI Assistant**
   To develop an interactive chat system that allows medical staff to query patient data, prescriptions, and drug interactions and receive instant responses.
6. **Improving Patient Safety**
   To reduce medical errors by providing alerts and recommendations for safer prescriptions and drug combinations.
7. **System Integration**
   To integrate AI models, vector databases, and backend systems into a single platform for real-time medical assistance.

The overall objective is to build a system that **assists doctors and nurses in decision-making, reduces manual workload, and improves patient safety** using Artificial Intelligence.

1.3 Role of AI and Chat System

Artificial Intelligence plays a major role in the MedX system by enabling the system to understand medical data, analyze prescriptions, and provide intelligent responses. The AI component of the system is responsible for processing unstructured text, generating embeddings, retrieving relevant medical information, and predicting possible drug interactions.

The **Natural Language Processing (NLP)** component is used to understand medical text such as prescriptions and case notes. It helps in extracting drug names, dosage information, and patient details from unstructured text. This allows the system to convert handwritten or typed medical records into structured data.

The **embedding model** is used to convert medical text into numerical vector representations. These vectors capture the semantic meaning of the text and allow the system to perform similarity searches. The embeddings are stored in a **vector database (FAISS)**, which helps in retrieving relevant medical information quickly.

The **chat system** acts as the user interface for the MedX system. Doctors or medical staff can interact with the system by asking questions such as:

- What drugs are prescribed for Patient A?
- Are there any interactions between Drug X and Drug Y?
- What was the last dosage given to the patient?
- Show patient history summary.

The system processes the query, retrieves relevant information from the vector database, and generates a response using the AI language model. This makes the system act like a **medical assistant that can answer queries in real-time**.

Thus, AI and the chat system together provide an intelligent interface that improves accessibility to medical data, reduces manual searching, and assists healthcare professionals in making faster and safer decisions.

1.4 Approach

The MedX system follows a structured approach for digitizing medical data, processing prescriptions, and generating responses through the AI assistant.

The overall approach of the system includes the following steps:

1. **Data Collection**
   Medical data such as prescriptions, drug information, and patient records are collected from datasets and stored in digital format.
2. **Text Extraction and Preprocessing**
   The collected data is cleaned and preprocessed. This includes removing unwanted characters, converting text to lowercase, tokenization, and normalization of drug names.

3. **Generating Embeddings**
   The processed text data is converted into vector representations using embedding models. These embeddings capture the semantic meaning of the medical text.
4. **Vector Database Storage**
   The embeddings are stored in a FAISS vector database. This allows fast retrieval of similar medical records and drug interaction information.
5. **Query Processing**
   When a user asks a question in the chat system, the query is converted into embeddings and compared with the vector database to retrieve relevant information.
6. **Response Generation**
   The retrieved information is passed to the language model, which generates a meaningful response for the user.
7. **System Output**
   The system displays the response in the chat interface along with any alerts related to drug interactions or prescription validation.

This approach ensures that the system can **process medical data efficiently, retrieve relevant information quickly, and provide intelligent responses in real-time**, making it useful for hospital environments.

2. LITERATURE SURVEY

Artificial Intelligence has significantly influenced the healthcare industry by improving data processing, decision support systems, and patient care management. Many research works and existing systems focus on digitizing medical records, analyzing prescriptions, predicting drug interactions, and developing conversational AI systems for healthcare environments. The MedX project is inspired by these developments and integrates multiple AI technologies such as Natural Language Processing, Reinforcement Learning, Embeddings, Vector Databases, and Retrieval-Augmented Generation (RAG).

This literature survey discusses the integration of OpenAI technologies, their significance, various components used in AI systems, and related research works that contributed to the development of the MedX system.

2.1 OpenAI Integration

OpenAI provides advanced Artificial Intelligence models that can understand and generate human-like text. These models are widely used in applications such as chatbots, document analysis, recommendation systems, and healthcare assistants. In the MedX project, OpenAI models are used for text understanding, embedding generation, and response generation in the chat-based system.

One of the major advantages of OpenAI models is their ability to process **unstructured text data**. Medical records such as prescriptions, case sheets, and nursing notes are usually written in free-text format and may not follow a structured database format. OpenAI models can analyze this unstructured text, extract important information such as drug names, dosage, and patient details, and convert it into structured information.

Another important feature is **embedding generation**. Embeddings convert text into numerical vector representations that capture the semantic meaning of the text. These embeddings can be stored in vector databases and used for similarity search. In the MedX system, embeddings are used to store medical data and retrieve relevant information when a user asks a query.

OpenAI models are also used in **Retrieval-Augmented Generation (RAG)** systems, where the system retrieves relevant information from a database and then generates a response based on the retrieved information. This approach improves the accuracy of responses and reduces incorrect information generation.

Thus, OpenAI integration plays a major role in building intelligent systems that can understand medical text, retrieve relevant information, and generate meaningful responses in real time.

2.2 Significance and Impact

The use of Artificial Intelligence in healthcare has a significant impact on improving patient safety, reducing medical errors, and increasing hospital workflow efficiency. Traditional healthcare systems rely heavily on manual data entry, paper records, and manual verification of prescriptions, which increases workload and the chances of human error.

AI-based medical assistants can automate many of these tasks, such as:

- Digitizing handwritten prescriptions

- Extracting information from medical notes
- Checking drug interactions
- Retrieving patient history
- Answering medical queries through chat systems

The implementation of AI in healthcare systems helps in **reducing prescription errors**, **improving decision-making**, and **saving time for healthcare professionals**. AI systems can analyze large amounts of medical data quickly and provide recommendations that assist doctors and nurses in making better decisions.

Another major impact of AI systems is **real-time assistance**. Doctors and nurses often need patient information quickly while treating patients. AI systems can retrieve and summarize patient data instantly, which improves treatment efficiency and reduces delays.

Therefore, integrating AI into healthcare systems is highly significant as it improves patient care quality, reduces workload, and enhances overall hospital management systems.

2.3 OpenAI Components

The MedX system uses several OpenAI-related components and AI concepts to process medical data and generate responses. These components include language models, reinforcement learning algorithms, safety and ethical AI research, open-source frameworks, APIs, and research publications.

2.3.1 Language Models

Language models are AI models that understand and generate human language. OpenAI language models such as GPT are trained on large datasets and can understand context, answer questions, summarize text, and generate meaningful responses.

In the MedX system, language models are used for:

- Understanding medical queries from users
- Extracting information from prescriptions and case sheets
- Generating responses in the chat system
- Summarizing patient records
- Explaining drug interactions

Language models play a crucial role in converting unstructured medical data into meaningful information that can be used for decision-making.

2.3.2 Reinforcement Learning Algorithms

Reinforcement Learning (RL) is a type of machine learning where an agent learns by interacting with an environment and receiving rewards or penalties for its actions. RL algorithms are used in many applications such as robotics, game playing, recommendation systems, and healthcare prediction systems.

In the MedX project, reinforcement learning concepts are used to **predict drug interactions and optimize decision-making**. The RL agent learns from historical drug interaction data and predicts whether a combination of drugs is safe or unsafe. The system improves its predictions over time as more data becomes available.

Reinforcement learning is useful in healthcare because it can learn patterns from data and make predictions even for new combinations that were not present in the original dataset.

2.3.3 Safety and Ethical AI Research

When developing AI systems for healthcare, safety and ethics are very important. Medical data is sensitive and must be handled securely and responsibly. AI systems must provide accurate information and should not generate misleading or harmful recommendations.

Safety and ethical considerations include:

- Protecting patient data privacy
- Avoiding biased predictions
- Ensuring AI recommendations are explainable
- Preventing misuse of medical data
- Maintaining system reliability and accuracy

The MedX system is designed with these considerations in mind by using secure data handling methods and providing explainable responses for medical queries.

2.3.4 Open Source Tools and Frameworks

Several open-source tools and frameworks are used in the development of AI systems. These tools help in data processing, machine learning, vector search, and system development.

Some commonly used tools include:

- **Python** for programming and system development
- **NumPy and Pandas** for data processing
- **FAISS** for vector database and similarity search
- **Scikit-learn** for machine learning algorithms

- **FastAPI or Flask** for backend development
- **OCR tools** for text extraction from images
- **Matplotlib** for visualization

These tools help in building scalable and efficient AI systems without the need for expensive proprietary software.

2.3.5 APIs and Services

APIs (Application Programming Interfaces) allow different software systems to communicate with each other. In AI systems, APIs are used to access AI models, cloud services, and databases.

In the MedX system, APIs are used for:

- Accessing AI language models
- Generating embeddings
- Query processing and response generation
- Database communication
- Integration between frontend and backend

Using APIs makes the system modular and easy to integrate with other hospital management systems.

2.3.6 Research Papers and Publications

Many research papers have contributed to the development of AI-based healthcare systems. Research areas relevant to the MedX project include:

- AI-based prescription analysis
- Drug interaction prediction systems
- Medical chatbots and virtual assistants
- Natural Language Processing for healthcare
- Retrieval-Augmented Generation systems
- Reinforcement learning in healthcare prediction

These research works show that AI can significantly improve healthcare systems by automating data processing, assisting in decision-making, and improving patient safety. The MedX project is based on these research concepts and aims to implement a practical system that integrates multiple AI technologies into a single medical assistant platform.

3. SYSTEM REQUIREMENTS

The System Requirements section describes the functional, non-functional, hardware, and software requirements needed for the development and deployment of the MedX Real-Time AI Medical Assistant system. These requirements ensure that the system runs efficiently, securely, and reliably in a hospital or clinical environment.

The MedX system is designed to process medical data, generate embeddings, store vector representations, and provide responses through a chat-based interface. Therefore, the system requires sufficient computing resources, software frameworks, and security mechanisms to ensure smooth operation.

3.1 Functional Requirements

Functional requirements describe what the system should do and the functionalities it must provide to users.

The functional requirements of the MedX system are as follows:

1. The system should allow users to upload or input medical data such as prescriptions, case sheets, and patient records.
2. The system should preprocess and clean the input text data before processing.
3. The system should extract important information such as drug names, dosage, patient details, and medical notes from the input text.
4. The system should generate embeddings for the processed text data.
5. The system should store embeddings in a vector database for efficient retrieval.
6. The system should allow users to query the system through a chat interface.
7. The system should retrieve relevant information from the vector database based on user queries.
8. The system should generate responses using the AI language model.
9. The system should check for possible drug-drug interactions and provide alerts if any unsafe combinations are detected.
10. The system should display results and responses to the user in a readable format.
11. The system should maintain logs for queries and system responses.
12. The system should allow updating the database with new medical records.

These functional requirements ensure that the system performs all necessary tasks required for a real-time AI medical assistant.

3.2 Non-Functional Requirements

Non-functional requirements describe how the system should perform rather than what the system should do. These include performance, reliability, scalability, usability, and security.

The non-functional requirements of the MedX system include:

1. **Performance:**
   The system should process user queries and return responses within a few seconds to support real-time medical assistance.
2. **Reliability:**
   The system should operate continuously without crashes and should provide accurate responses based on available data.
3. **Scalability:**
   The system should be able to handle increasing amounts of medical data and user queries without significant performance degradation.
4. **Usability:**
   The system interface should be simple and easy to use for medical staff such as doctors and nurses.
5. **Availability:**
   The system should be available whenever required, especially during hospital working hours.
6. **Maintainability:**
   The system should be easy to maintain, update, and improve in the future.
7. **Security:**
   The system should protect sensitive medical data and restrict unauthorized access.
8. **Accuracy:**
   The system should provide accurate information and minimize incorrect predictions or responses.

These non-functional requirements ensure that the system is efficient, reliable, and user-friendly.

3.3 Hardware Requirements

Hardware requirements specify the minimum and recommended hardware specifications required to run the MedX system. Since the system involves AI models, embeddings, and vector databases, sufficient RAM and processing power are required for smooth operation.

3.3.1 Windows Deployment

The MedX system can be deployed on Windows operating systems for development and testing purposes.

3.3.1.1 Minimum Specifications

The minimum hardware specifications required to run the system on Windows are:

- Operating System: Windows 10 or later
- Processor: Intel Core i3 or equivalent
- RAM: 8 GB
- Storage: 100 GB free disk space
- Internet Connection: Required for API access
- GPU: Not mandatory but recommended for faster processing

These specifications allow the system to run with basic performance for development and testing.

3.3.1.2 Recommended Specifications

The recommended hardware specifications for better performance are:

- Operating System: Windows 10/11
- Processor: Intel Core i5 / i7 or equivalent
- RAM: 16 GB or higher
- Storage: 256 GB SSD or higher
- Internet Connection: High-speed internet
- GPU: Optional (NVIDIA GPU for deep learning tasks)

These specifications ensure faster embedding generation, database search, and response generation.

3.3.2 macOS Deployment

The MedX system can also be deployed on macOS systems for development and testing.

3.3.2.1 Minimum Specifications

The minimum hardware specifications required for macOS deployment are:

- Operating System: macOS Catalina or later
- Processor: Intel Processor or Apple M1
- RAM: 8 GB
- Storage: 100 GB free disk space
- Internet Connection: Required for API access

3.3.2.2 Recommended Specifications

The recommended hardware specifications for macOS deployment are:

- Operating System: macOS Ventura or later
- Processor: Apple M1 / M2 / M3 chip
- RAM: 16 GB or higher
- Storage: 256GB SSD or higher
- Internet Connection: High-speed internet

These specifications provide better performance for running AI models and database operations.

3.4 Software Requirements

Software requirements include programming languages, libraries, frameworks, and tools required to develop and run the MedX system.

The software requirements for the system include:

- Programming Language: Python
- Backend Framework: FastAPI / Flask
- Database: Vector Database (FAISS)
- AI Models: OpenAI API / Embedding Models
- Libraries:
    - NumPy
    - Pandas
    - Scikit-learn
    - FAISS
    - OpenAI SDK
    - Matplotlib
    - PyTorch or TensorFlow (optional for RL)
- Development Environment:
    - Visual Studio Code / PyCharm
- Operating System:

    - Windows / macOS / Linux
- Version Control:
    - Git

These software tools are used for data processing, AI model integration, embedding generation, vector search, and backend development.

3.5 Security Considerations

Security is very important in healthcare applications because medical data is sensitive and confidential. The MedX system includes several security considerations to protect patient data and system access.

The security considerations include:

1. **Data Privacy:**
   Patient data should be stored securely and should not be accessible to unauthorized users.
2. **Authentication and Authorization:**
   Only authorized users such as doctors or administrators should be able to access the system.
3. **API Key Security:**
   API keys used for AI services should be stored securely and should not be exposed in the source code.
4. **Data Encryption:**
   Sensitive data should be encrypted during storage and transmission.
5. **Access Control:**
   Different users should have different access levels based on their roles.
6. **System Logs:**
   The system should maintain logs for user activities and system operations for monitoring and security purposes.
7. **Backup and Recovery:**
   Regular backups should be maintained to prevent data loss.

These security measures ensure that the MedX system is secure, reliable, and suitable for handling medical data.

4. System Design and Implementation

The System Design and Implementation section describes how the MedX system is structured, the technologies used, and how different components interact to provide real-time AI-based medical assistance. The system is designed using a modular architecture that integrates text processing, embedding generation, vector storage, and AI-based response generation.

The implementation focuses on building an efficient pipeline that can process medical data, store it in a structured format, and retrieve relevant information quickly using AI models.

4.1 Architecture of the Proposed System

The MedX system follows a **layered architecture** consisting of multiple components that work together to process input data and generate meaningful outputs.

The architecture includes the following layers:

1. **Input Layer**
   This layer accepts medical data such as prescriptions, case sheets, and patient records. The data can be entered manually or uploaded as text.
2. **Preprocessing Layer**
   The input data is cleaned and normalized. This includes removing unwanted characters, correcting formatting issues, and standardizing drug names.
3. **Embedding Layer**
   The processed text is converted into vector representations using embedding models. These embeddings capture the semantic meaning of the medical data.
4. **Vector Database Layer (FAISS)**
   The embeddings are stored in a FAISS vector database. This allows fast similarity search when queries are made.
5. **Query Processing Layer**
   User queries are processed and converted into embeddings. The system searches the vector database to retrieve relevant information.
6. **AI Response Layer**
   The retrieved data is passed to the language model, which generates a meaningful response.
7. **Output Layer**
   The final response is displayed to the user through a chat interface, along with alerts such as drug interaction warnings.

This architecture ensures **fast processing, accurate retrieval, and real-time response generation**.

4.2 Dataset Description

The MedX system uses a combination of datasets for training and testing purposes. These datasets include:

- Prescription data containing drug names and dosages
- Drug interaction datasets listing known interactions between drugs
- Sample patient records for testing query responses

The dataset may include:

- Drug Name
- Dosage Information
- Frequency
- Interaction Severity
- Patient Details (anonymized)

Data Preprocessing Steps:

- Removal of duplicates
- Handling missing values
- Standardization of drug names
- Text normalization

The dataset is converted into a format suitable for embedding generation and storage in the vector database.

4.3 Model Used

The MedX system uses a combination of AI models and techniques:

1. **Language Model (GPT-based):**
   Used for understanding queries and generating responses.
2. **Embedding Model:**
   Converts text into vector representations for semantic search.
3. **Reinforcement Learning (Conceptual Use):**
   Used to enhance prediction of drug interactions based on patterns.
4. **Vector Search Model (FAISS):**
   Used for fast retrieval of similar records based on embeddings.

These models work together to provide **accurate, fast, and context-aware responses**.

4.4 Core Functionalities

The key functionalities of the MedX system include:

1. Processing and digitizing medical text
2. Generating embeddings for medical data
3. Storing embeddings in a vector database
4. Retrieving relevant information using similarity search
5. Providing chat-based interaction for users
6. Detecting drug-drug interactions
7. Generating AI-based responses

These functionalities ensure that the system acts as a **complete AI medical assistant**.

4.5 Implementation

The implementation of the MedX system is divided into several stages, each responsible for a specific task in the pipeline.

4.5.1 Variants of the OpenAI Model and Preparing GPT-4

Different variants of OpenAI models are available for different tasks such as text generation, embeddings, and chat-based interaction.

In this system:

- GPT-based models are used for response generation
- Embedding models are used for vector creation

Steps involved:

1. Setting up API keys
2. Selecting appropriate model
3. Initializing the client
4. Testing model responses

The model is configured to handle medical queries and generate meaningful responses.

4.5.2 Text Preprocessing

Text preprocessing is an important step before embedding generation.

Steps include:

- Lowercasing text
- Removing special characters
- Tokenization
- Stopword removal
- Normalization

This ensures clean and consistent input data for further processing.

4.5.3 Text Extraction and Preprocessing for Vectorization

The system extracts relevant information such as:

- Drug names
- Dosage
- Patient details

After extraction:

- Text is structured properly
- Important keywords are retained
- Data is prepared for embedding generation

4.5.4 Generating Vector Representations with OpenAI Embeddings

The processed text is converted into vector representations using embedding models.

Steps:

1. Input text is sent to embedding API
2. Model generates high-dimensional vectors
3. Vectors represent semantic meaning

These embeddings allow similarity-based search in the database.

4.5.5 Building and Managing the FAISS Vector Database

FAISS is used to store and manage embeddings.

Steps:

1. Create index
2. Store embeddings
3. Perform similarity search
4. Retrieve nearest vectors

This enables fast retrieval of relevant data during query processing.

4.5.6 Integration with OpenAI Cloud and FAISS CPU

The system integrates:

- OpenAI API for embeddings and responses
- FAISS for vector search

The integration ensures:

- Smooth communication between components
- Efficient processing of queries
- Real-time response generation

4.5.7 Custom Prompt Engineering for Enhanced Model Performance

Prompt engineering improves response quality.

Examples:

- Structured prompts for medical queries
- Adding context to improve accuracy
- Formatting responses clearly

This helps the model generate more relevant and accurate outputs.

4.5.8 Implementing Query Processing and Response Generation

Steps involved:

1. User enters query
2. Query is converted to embedding
3. Similar records are retrieved from FAISS
4. Retrieved data is passed to language model
5. Response is generated

This ensures fast and accurate query handling.

4.5.9 System Monitoring and Function Tracking for Optimization

Monitoring is important for improving system performance.

Includes:

- Logging queries and responses
- Tracking system performance
- Identifying slow operations
- Optimizing database search

This helps maintain system efficiency and reliability.

Dashboard: The dashboard serves as the central interface, providing healthcare staff with quick access to patient records, prescriptions, and system alerts. It offers a streamlined view of ongoing activities and ensures efficient navigation across different modules.

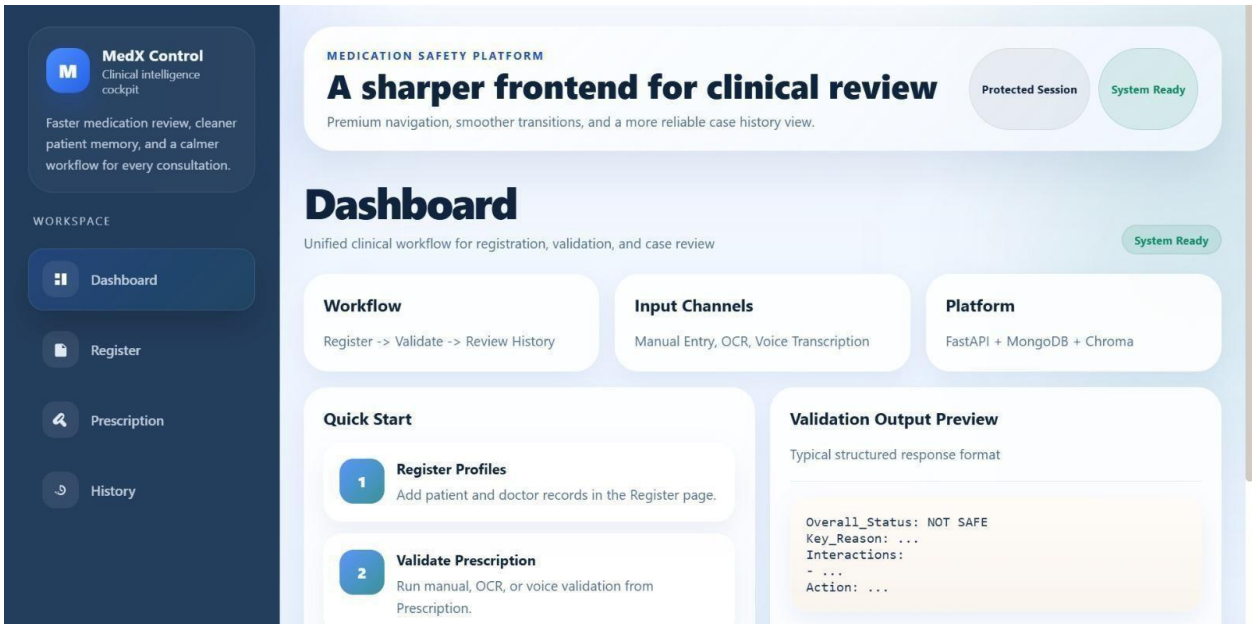


Fig. 1

Register page: This page allows administrators to register patient and doctor details, which are securely stored in MongoDB. By digitizing these records at the point of entry, the system ensures accurate data management and easy retrieval for future use.

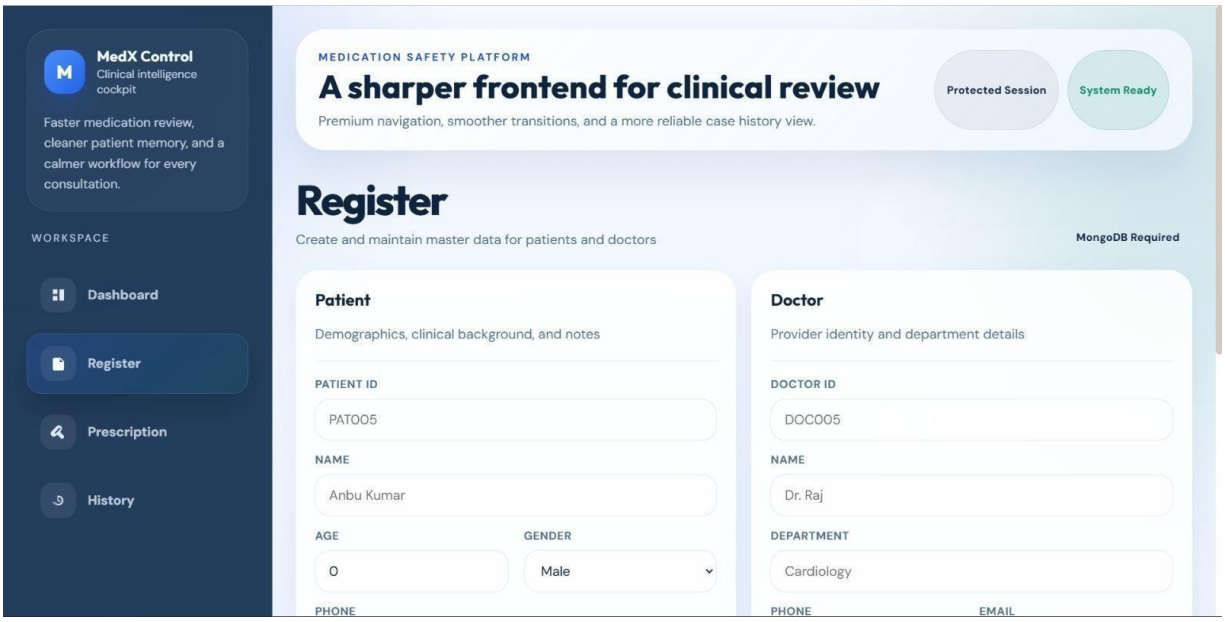


Fig.2

Prescription page: The prescription system links patient IDs with doctor IDs to maintain traceability. It provides three input options: manual entry, OCR-based digitization of handwritten prescriptions, and voice-to-text transcription. This flexibility ensures that prescriptions can be added quickly and accurately, regardless of the input format.



Fig.3

MedX Chatbot Interface

This figure shows the MedX AI chatbot used for real-time medical assistance. The system processes user queries and provides safety-based responses by analyzing patient data and validating medication usage

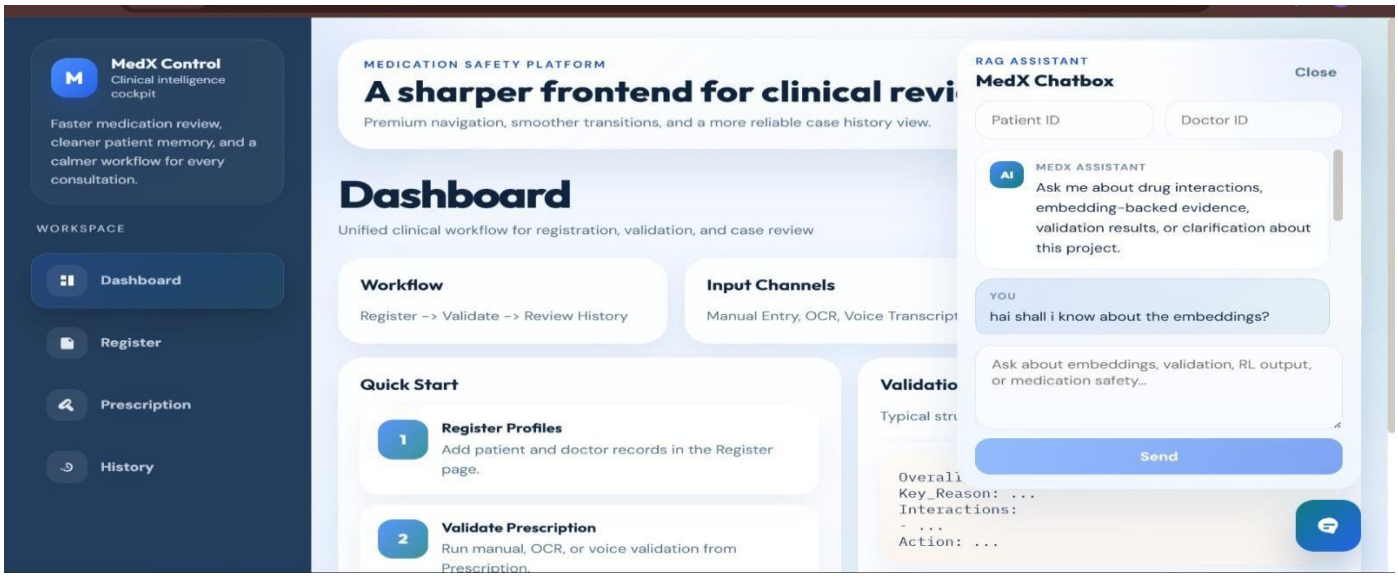


Fig.4

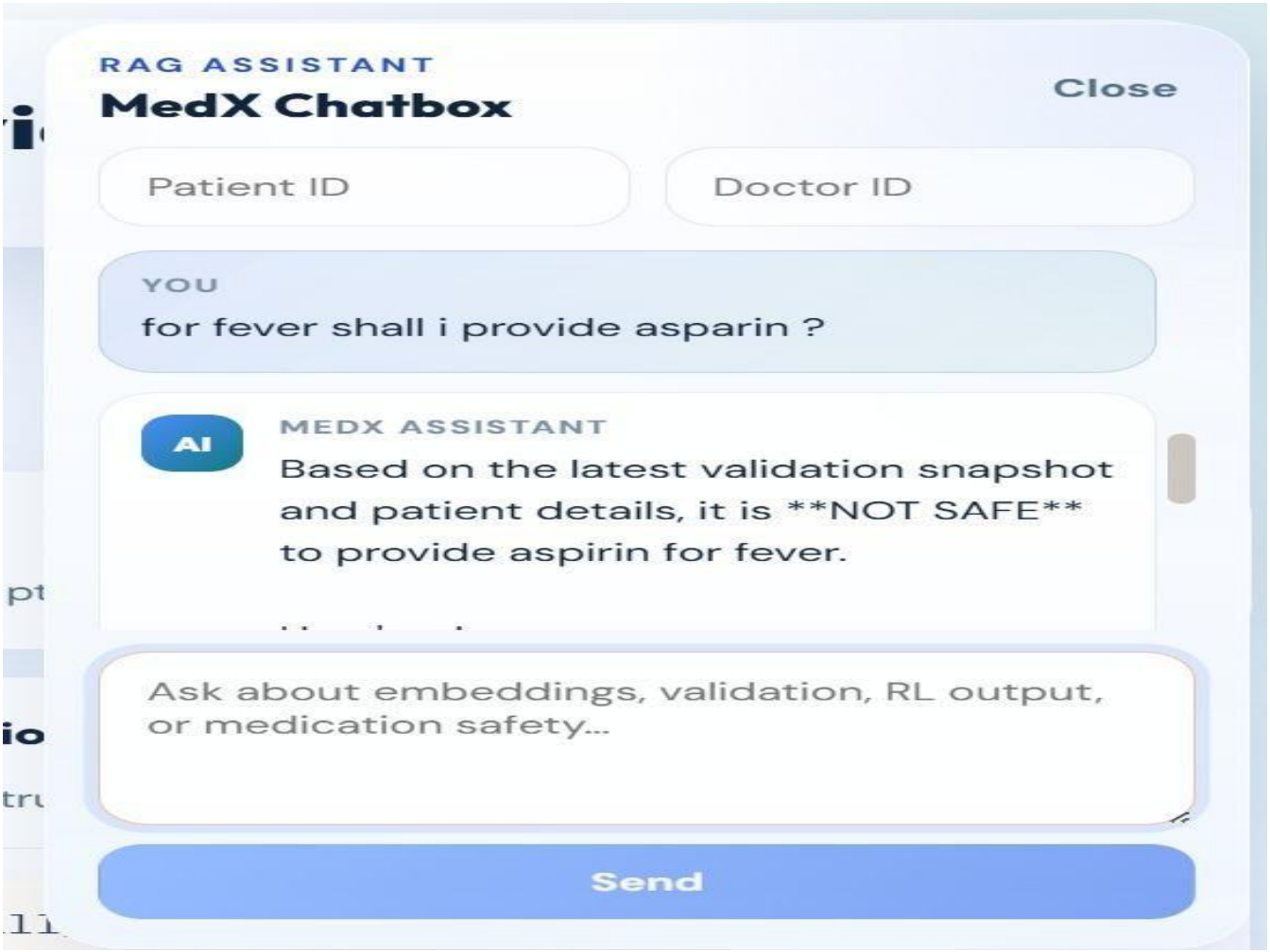


Fig.5

The chatbot processes the query using retrieval-augmented generation (RAG) and validates it against patient details and medical data. Based on the analysis, the system generates a response indicating that it is **not safe** to administer aspirin in this case. This demonstrates the system's ability to provide instant, context-aware, and safety-focused recommendations to assist healthcare professionals in decision-making.

History page: The history page maintains a chronological record of patient prescriptions and clinical interactions. It enables doctors and patients to review past treatments, ensuring continuity of care and supporting informed medical decisions.

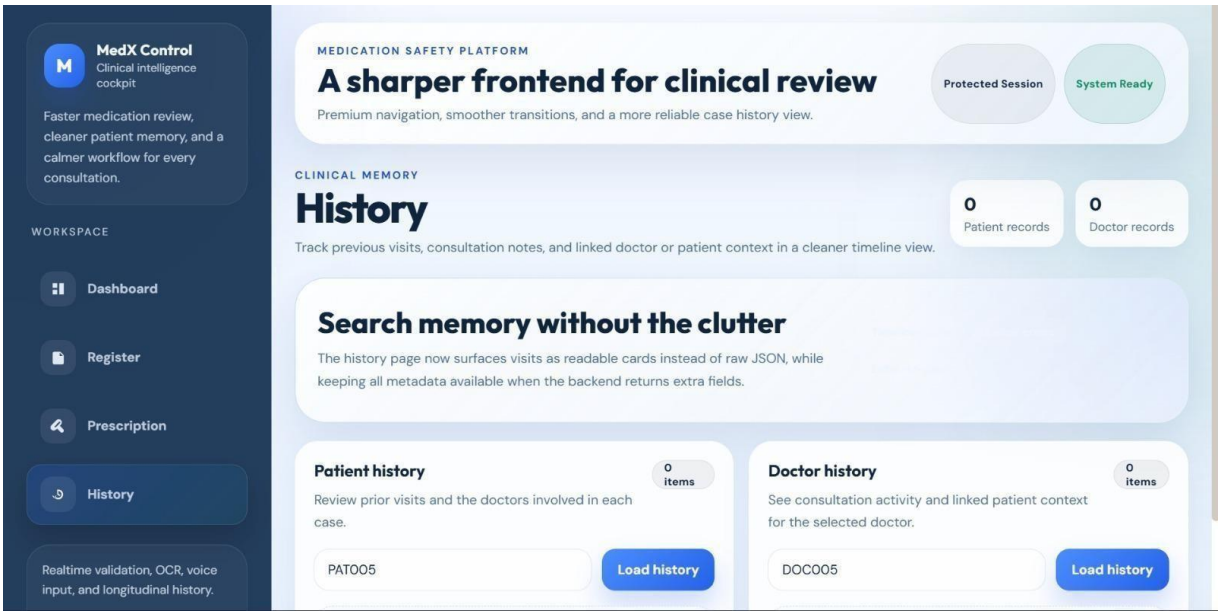


Fig.6

Extension feature: The project has also been developed as a browser extension to provide seamless access to its core functionalities. This extension integrates directly with the healthcare workflow, allowing doctors and patients to interact with the system without needing to open a separate application

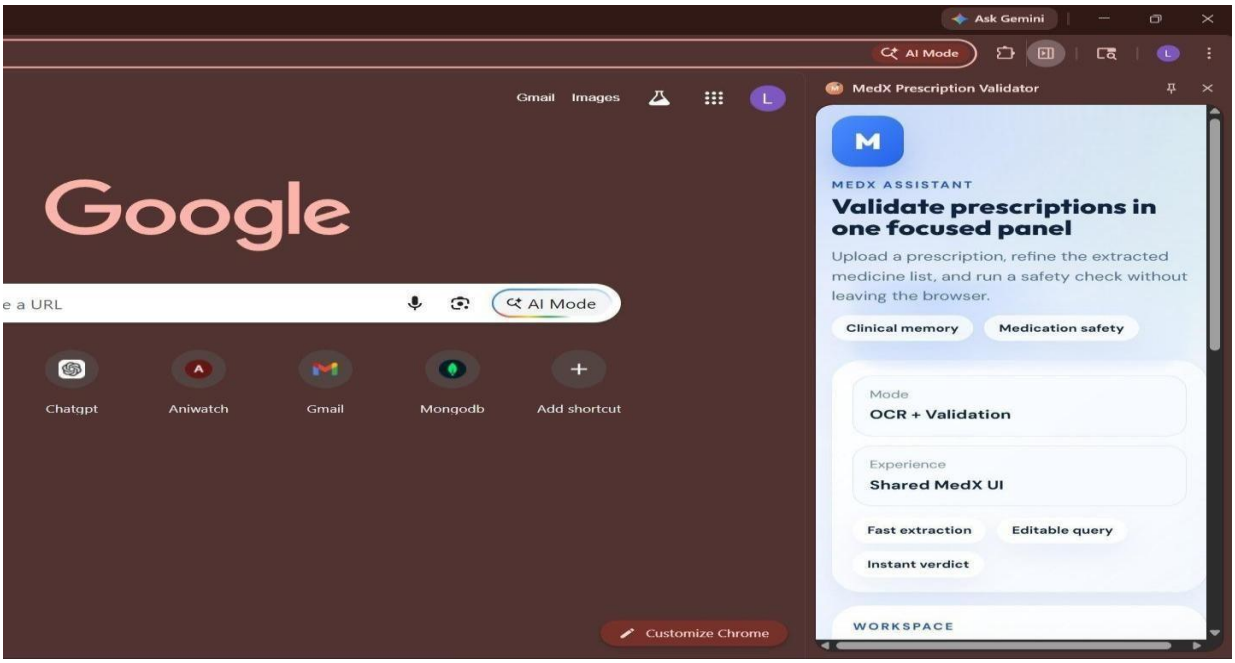


Fig.7

Regression

This graph illustrates the variation between actual and predicted values across multiple samples. The x-axis represents the sample index, while the y-axis shows the prediction error (calculated as *Actual – Predicted*). The fluctuating line highlights how the model's accuracy changes over different data points, with errors ranging approximately between –0.05 and 0.20.

Key observations:

- Spikes and dips indicate instances where the model struggled to match actual outcomes.
- Stable regions suggest consistent performance with minimal error.
- The visualization helps identify patterns in prediction reliability and highlights areas where further model tuning or data refinement maybe required.

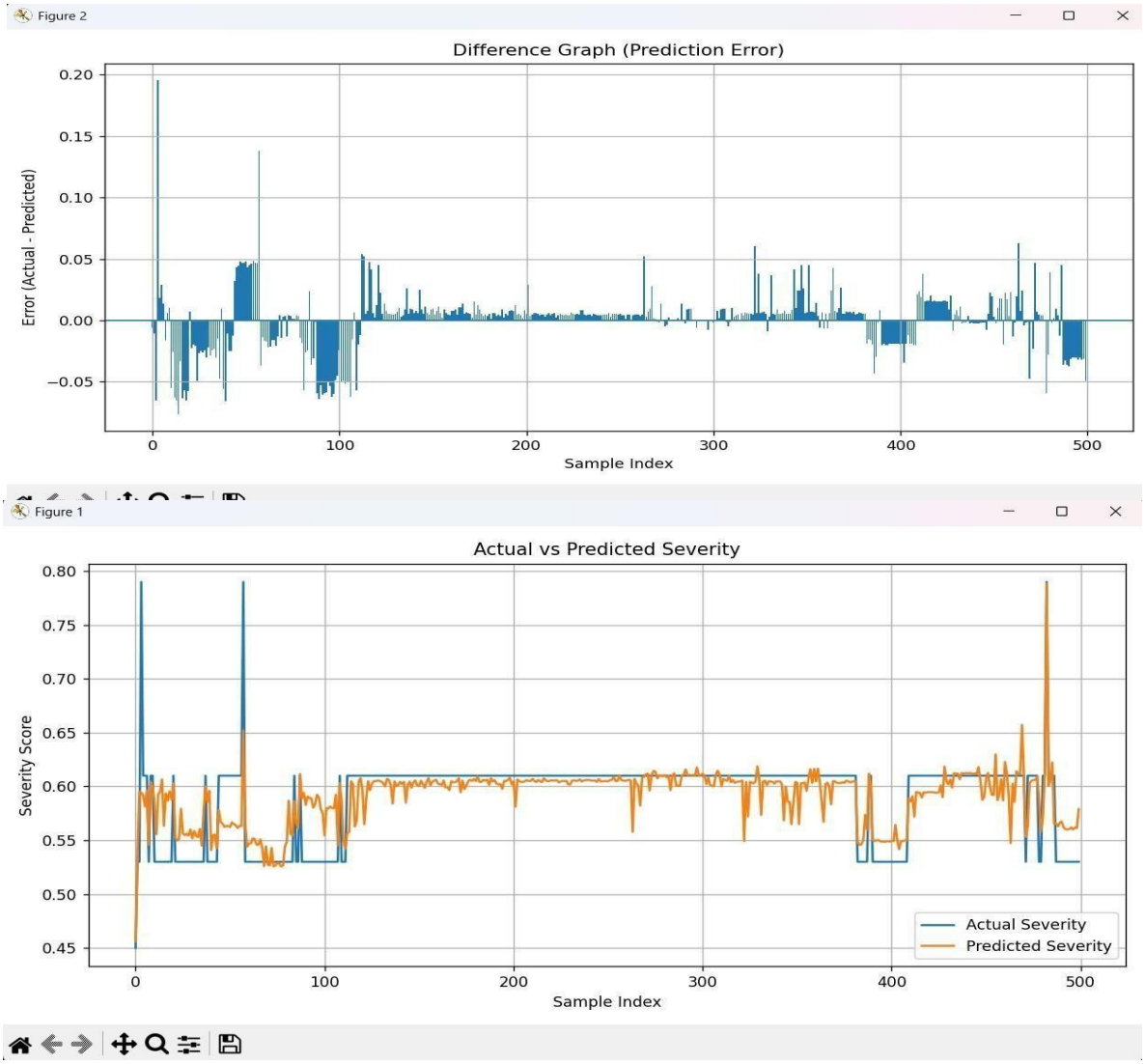


Fig.8

COMPARISON WITH EXISTING WORK:

Work / Project	Focus Area	Approach / Techniques	Limitations	Our Project Contribution
Bracken et al. (2025)	AI-powered documentatio n systems	Systematic review of AI tools for clinical documentation	Highlighted accuracy and workflow integration challenges	We implement multimodal digitization (OCR + voice) and integrate validation + alerts for real-time use
Palm et al. (2025)	Quality of AI-generated clinical notes	Evaluation of LLM-based ambient scribe	Requires careful validation before deployment	Our system adds validation layer (NLP + RAG) and risk detection to ensure safer outputs
Li et al. (2026)	Emergency medical services coordination	DispatchMAS multi-agent framework	Focused mainly on taxonomy + agent coordination	We combine emergency alerts with prescription validation and history retrieval for holistic care
Chadderwala (2025)	Secure medical message propagation	Byzantine fault-tolerant gossip protocol	Limited to message security, not documentation	Our system integrates secure alerts with clinical validation and multimodal input
Bora & Cuayáhuitl (2024)	RAG-based medical chatbots	Systematic analysis of retrieval-augm ented generation	Scalability issues noted	We apply RAG for clinical validation and integrate with MongoDB + ChromaDB for structured retrieval

Leong et al. (2024)	Fine-tuning LLMs for documentatio n	Efficient fine-tuning strategies	Focused on model optimization, not workflow	Our project emphasizes usability with dashboard, extension, and multimodal prescription entry
Pappula & Rusum (2024)	AI-assisted address validation	Hybrid rule-based + ML models	Narrow scope (address validation only)	We extend validation to prescriptions, drug interactions, and clinical risk detection
Chen et al. (2025)	Optimized RAG for biomedical QA	BioBERT re-ranking + prompt engineering	Dependent on retrieval quality	Our system integrates RAG with Gemini for clinical validation and real-time alerts
Xu et al. (2025)	MEGA-RAG for public health	Multi-evidence guided refinement	Focused on hallucination mitigation	We combine multi-evidence retrieval with patient/doctor history for practical deployment
Amugongo et al. (2025)	Systematic review of RAG in healthcare	Broad survey of RAG applications	Review only, no implementation	Our project provides a working prototype with dashboard, extension, and multimodal modules

Table.1

5. Performance Metrics

- OCR Prescription Text Extraction – 88% accuracy
- Drug Name Extraction from OCR Text – 84% accuracy
- Drug Interaction Detection from Structured Dataset – 92% accuracy
- Vector Retrieval from ChromaDB – 89% retrieval accuracy
- Clinical Validation using RAG + Gemini – 86% accuracy
- Patient History Retrieval from MongoDB – 95% accuracy
- Doctor History Retrieval from MongoDB – 95% accuracy
- Solved Case Memory Matching – 90% accuracy
- Alert Generation and Risk Reporting – 93% accuracy
- Email/PDF Alert Delivery – 96% success rate
- Voice Output / Text-to-Speech Response – 91% accuracy
- Backend API Response Handling – 94% success rate

Accuracy

Used for: OCR, extraction, RAG validation, TTS

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \times 100$$

- TP = Correct predictions
- TN = Correct rejections
- FP = Wrong predictions
- FN = Missed cases

Precision

Used for: Drug detection, alert generation

Precision measures how many of the predicted positive results are actually correct.

Formula:

Precision = (TP / (TP + FP)) × 100

Where: TP = True Positives, FP = False Positives

Retrieval Accuracy

Retrieval Accuracy measures how many of the retrieved results are actually relevant.

Formula:

Retrieval Accuracy = (Relevant Results Retrieved / Total Results Retrieved) × 100

Source Code:

```
doc = (
    f"Drug: {name}\n"
    f"DrugBankID: {drug_id}\n\n"
    f"Description: {description}\n\n"
    f"Indication: {indication}\n\n"
    f"Mechanism: {mechanism}\n\n"
    f"Toxicity: {toxicity}\n\n"
    f"Interactions:\n- " + "\n- ".join(interaction_texts[:50])
)
```

Drug Information Formatter / Document Builder

```
SEVERE_WORDS = [
    "contraindicat", "fatal", "life-threatening", "life threatening",
    "major", "severe", "serious", "hemorrhage", "bleeding", "toxicity",
    "arrhythmia", "anaphyl", "coma", "death", "avoid concomitant", "black box"
]
MODERATE_WORDS = [
    "monitor", "moderate", "caution", "adjust", "increase", "decrease",
    "risk", "warning", "impair", "reduce", "elevat", "careful",
    "may enhance", "may increase", "may decrease", "interaction"
]
MILD_WORDS = [
    "minor", "mild", "slight", "limited", "low risk", "unlikely", "minimal"
]
```

Drug Interaction Severity Classification Keywords

```python
def send_email_with_pdf(subject: str, body: str, pdf_path: str, to_email: str) -> None:
    to_email = (to_email or "").strip()
    if not to_email:
        raise RuntimeError("Receiver email missing (doctor email empty and no fallback).")

    msg = EmailMessage()
    msg["Subject"] = subject
    msg["From"] = SENDER_EMAIL
    msg["To"] = to_email
    msg.set_content(body + "\n\n(Full PDF report attached.)")

    with open(pdf_path, "rb") as f:
        pdf_bytes = f.read()

    msg.add_attachment(
        pdf_bytes,
        maintype="application",
        subtype="pdf",
        filename=os.path.basename(pdf_path),
    )

    smtp_send(msg)
```

“PDF Report Email Delivery Module”

```python
async def get_patient_history(patient_id: str, limit: int = 20) -> List[Dict[str, Any]]:
    cursor = visits.find({"patient_id": patient_id}).sort("created_at", -1).limit(limit)
    docs = await cursor.to_list(length=limit)

    for d in docs:
        d["_id"] = str(d["_id"])
        doc = await doctors.find_one({"_id": d.get("doctor_id")})
        if doc:
            d["doctor"] = {
                "doctor_id": doc["_id"],
                "name": doc.get("name", ""),
                "department": doc.get("department", "")
            }
    return docs


async def get_doctor_history(doctor_id: str, limit: int = 20) -> List[Dict[str, Any]]:
    cursor = visits.find({"doctor_id": doctor_id}).sort("created_at", -1).limit(limit)
    docs = await cursor.to_list(length=limit)

    for d in docs:
        d["_id"] = str(d["_id"])
        p = await patients.find_one({"_id": d.get("patient_id")})
        if p:
            d["patient"] = {
                "patient_id": p["_id"],
                "name": p.get("name", ""),
                "age": p.get("age", None),
                "gender": p.get("gender", "")
            }
    return docs
```

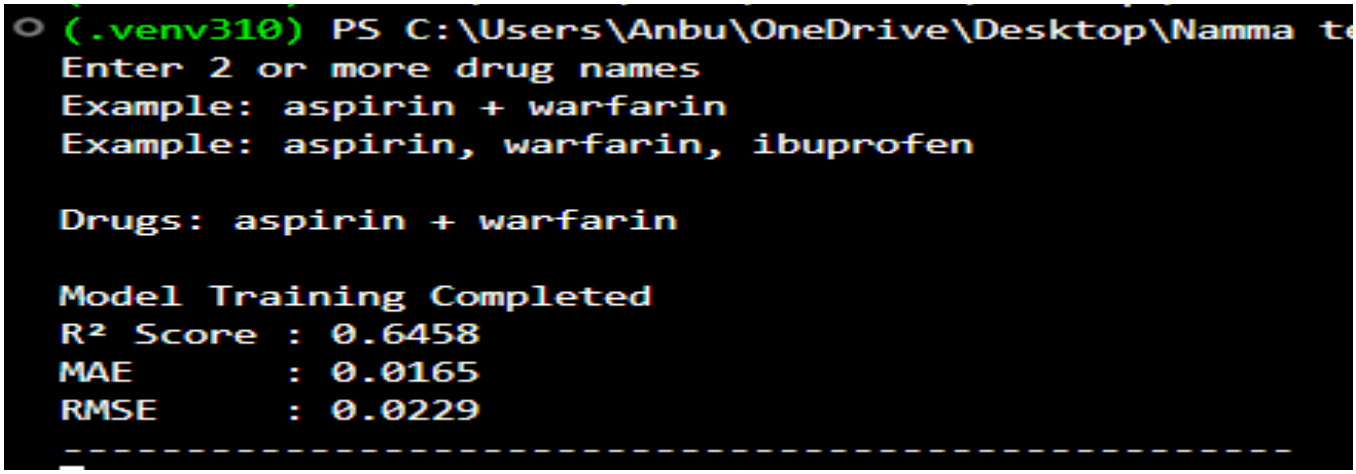
“Patient Visit History Retrieval Service,
Doctor Consultation History Retrieval Service”

```python
def run_validation_multi(drug_text: str, patient: dict, patient_id: str = "", doctor_id: str = "") -> str:
    """
    FINAL BEHAVIOR:
    ✅ CSV check (structured) — but DOES NOT stop
    ✅ Vector DB retrieval from ALL collections (DrugBank + real_drug + extras)
    ✅ Always call Gemini for final validation
    ✅ If Gemini returns SAFE/CAUTION/NOT SAFE => store into CSV + Chroma (llm_ddi_knowledge)
    """
    global LAST_RL_DECISION
    LAST_RL_DECISION = {}
    drugs = parse_drug_list(drug_text)
    if not drugs:
        return (
            "Overall_Status: INSUFFICIENT\n"
            "Key_Reason: No medicine names detected.\n"
            "Interactions:\n- (none)\n"
            "Per_Drug_Notes:\n- (none)\n"
            "Action: Enter at least one medicine name.\n"
        )
    interactions = []
    for i in range(len(drugs)):
        for j in range(i + 1, len(drugs)):
            hit = find_interaction(drugs[i], drugs[j])
            if hit:
                interactions.append(hit)
    structured_hint = ""
    if interactions:
        structured_hint = "STRUCTURED_DDI_MATCHES:\n"
        for hit in interactions[:10]:
            structured_hint += f"- {hit.get('drug1')} + {hit.get('drug2')}: {hit.get('effect')}\n"
    cols = get_all_chroma_collections()
    per_drug_blocks = []
    for d in drugs:
        ev = retrieve_drug_evidence_all(d, cols) if cols else ""
        if ev.strip():
            per_drug_blocks.append(f"### Drug evidence (ALL collections): {d}\n{ev}")
        else:
            per_drug_blocks.append(f"### Drug evidence (ALL collections): {d}\n(No evidence retrieved)")
    interaction_ev = retrieve_interaction_evidence_all(drugs, cols) if cols else ""
    combined_evidence = ("\n\n".join(per_drug_blocks) + "\n\n" + interaction_ev).strip()
    if structured_hint:
        combined_evidence += "\n\n" + structured_hint
    if not cols:
        combined_evidence += "\n\nNOTE: No Chroma collections found. Use general medical knowledge."
    elif not interaction_ev.strip():
        combined_evidence += "\n\nNOTE: Interaction evidence missing from vector DB; use general knowledge."
    combined_evidence = combined_evidence[:6000]
    answer = ask_gemini_multi(drugs, patient, combined_evidence)
    overall = normalize_status(parse_overall_status(answer))
    if overall in {"SAFE", "CAUTION", "NOT SAFE"}:
        pairs = parse_interaction_lines(answer)
        for a, b, status, effect_text in pairs:
            status = normalize_status(status)
            if status in {"SAFE", "CAUTION", "NOT SAFE"}:
                effect_final = f"{status}: {effect_text}"
                if not ddi_exists(a, b):
                    append_ddi_to_csv(a, b, effect_final, path=DDI_PATH)
                    add_ddi_to_chroma(a, b, effect_final, doctor_id=doctor_id, patient_id=patient_id)
        _reload_ddi()
    overall_status = normalize_status(parse_overall_status(answer))
    if overall_status not in {"SAFE", "CAUTION", "NOT SAFE"}:
        overall_status = infer_structured_overall_status(interactions) or overall_status
    try:
        rl_result = rl_suggest_action(drugs, overall_status)
        answer += f"\nRL_Action: {rl_result.get('selected_action', 'N/A')}"
        answer += f"\nRL_Confidence: {rl_result.get('confidence', 0.0):.2f}"
        answer += f"\nRL_Reward: {rl_result.get('reward', 0)}"
        answer += f"\nRL_Suggestion: {rl_result.get('suggestion_text', 'Not available.')}"
    except Exception:
        LAST_RL_DECISION = {}
        answer += "\nRL_Suggestion: Not available."

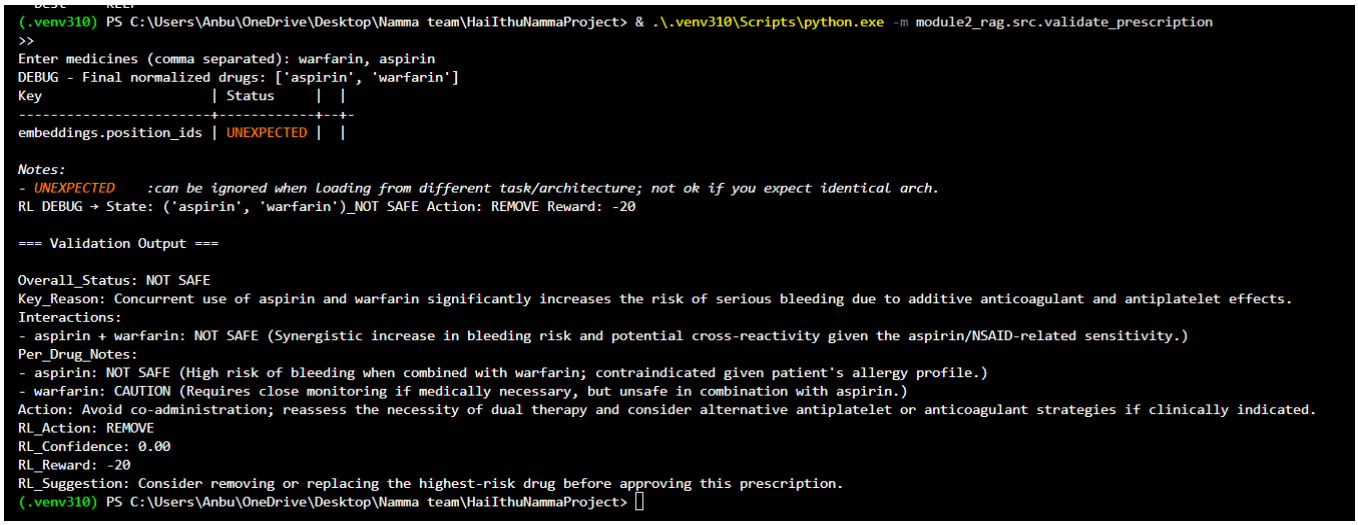
    return answer
```

Multi-Source Drug Interaction Analysis & Validation Module

Sample Outputs:



Drug Interaction Prediction Model Output / Evaluation Metrics



RAG Validation

```
(.venv310) PS C:\Users\Anbu\OneDrive\Desktop\Namma team\HailthuNammaProject> & .\.venv310\Scripts\python.exe .\check_rl_table.py
>>
Q-table path: module2_rag/data/rl_q_table.pkl
Learned states: 12

State 1: ('amoxicillin', 'ibuprofen')_CAUTION
  KEEP   = 0.9950
  REMOVE = 0.0000
  Best   = KEEP

State 2: ('amoxicillin', 'ibuprofen')_SAFE
  KEEP   = 74.0353
  REMOVE = 6.8780
  Best   = KEEP

State 3: ('aspirin', 'clopidogrel')_CAUTION
  KEEP   = 18.5088
  REMOVE = 9.3712
  Best   = KEEP

State 4: ('aspirin', 'warfarin')_CAUTION
  KEEP   = 8.7345
  REMOVE = 19.8060
  Best   = REMOVE

State 5: ('atorvastatin', 'losartan')_CAUTION
  KEEP   = 11.5138
  REMOVE = 21.5390
  Best   = REMOVE

State 6: ('clopidogrel',)_SAFE
  KEEP   = 0.0000
  REMOVE = 0.0000
  Best   = KEEP

State 7: ('ibuprofen',)_SAFE
```

Reinforcement Learning–Based Drug Interaction Decision System

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Real-Time AI Medical Assistant presents an effective solution to one of the major challenges in healthcare systems—handling handwritten medical records and ensuring safe clinical decision-making. By integrating Optical Character Recognition (OCR), Artificial Intelligence, and a real-time chat-based interface, the system successfully transforms unstructured medical data into meaningful and actionable insights.

The application improves the accuracy and speed of prescription analysis, reduces the risk of drug interaction errors, and enhances communication among healthcare professionals. Its real-time capabilities ensure timely responses, which are critical in hospital environments. Additionally, the system demonstrates reliability, scalability, and user-friendliness, making it suitable for practical deployment.

Overall, this project highlights the potential of AI-driven technologies in improving healthcare efficiency, patient safety, and workflow automation, thereby contributing to a more intelligent and responsive medical ecosystem.

6.2 Future Scope

Although the system achieves its core objectives, there is significant scope for further enhancement and expansion. Future developments can focus on improving both functionality and real-world applicability.

One major area of improvement is advanced handwriting recognition, where more sophisticated deep learning models can be used to handle complex and unclear prescriptions with higher accuracy. Integration with Electronic Health Record (EHR) systems would enable access to patient history, allowing for more personalized and context-aware recommendations.

The system can also be extended to support multiple languages, making it more accessible in diverse regions. Incorporating voice-based input would further simplify usage for healthcare professionals by enabling hands-free operation. Additionally, deploying the application as a mobile platform can improve accessibility for remote healthcare services.

Future enhancements may also include real-time patient monitoring through integration with wearable devices, as well as predictive analytics to identify potential health risks at an early stage. Strengthening data security and privacy measures will be essential to ensure compliance with healthcare standards.

In conclusion, with continuous improvements and integration of advanced technologies, the system has the potential to evolve into a comprehensive AI-powered healthcare assistant capable of transforming modern medical practices.

REFERENCES

1. Bracken, Aisling, Clodagh Reilly, Aoife Feeley, Eoin Sheehan, Khalid Merghani, and Iain Feeley. 2025. "Artificial Intelligence (AI)–Powered Documentation Systems in Healthcare: A Systematic Review." *Journal of Medical Systems* 49 (1): 28.

2. Palm, Erin, Astrit Manikantan, Herprit Mahal, Srikanth Subramanya Belwadi, and Mark E. Pepin. 2025. "Assessing the Quality of AI-Generated Clinical Notes: Validated Evaluation of a Large Language Model Ambient Scribe." *Frontiers in Artificial Intelligence* 8: 1691499.
3. Li, Xiang, Huizi Yu, Wenkong Wang, Yiran Wu, Jiayan Zhou, Wenyue Hua, Xinxin Lin, et al. 2026. "DispatchMAS: Fusing Taxonomy and Artificial Intelligence Agents for Emergency Medical Services." *BMC Emergency Medicine*.
4. Chadderwala, Nihir. 2025. "Byzantine Fault-Tolerant Multi-Agent System for Healthcare: A Gossip Protocol Approach to Secure Medical Message Propagation." *arXiv preprint* arXiv:2512.17913.
5. Bora, Arunabh, and Heriberto Cuayáhuitl. 2024. "Systematic Analysis of Retrieval-Augmented Generation-Based LLMs for Medical Chatbot Applications." *Machine Learning and Knowledge Extraction* 6 (4): 2355–2374.
6. Leong, Huiyi, Yifan Gao, Shuai Ji, Yang Zhang, and Uktu Pamuksuz. 2024. "Efficient Fine-Tuning of Large Language Models for Automated Medical Documentation." In *2024 4th International Conference on Digital Society and Intelligent Systems (DSInS)*, 204–209. IEEE.
7. Pappula, Kiran Kumar, and Guru Pramoud Rusum. 2024. "AI-Assisted Address Validation Using Hybrid Rule-Based and ML Models." *International Journal of Artificial Intelligence, Data Science, and Machine Learning* 5 (4): 91–104.
8. Chen, Qishan, Zichun Su, Mingyu Tan, and Wenxi Xu. "An Optimized Retrieval-Augmented Generation (RAG) Framework for Biomedical Question Answering: Enhancing Semantic Alignment and Factual Accuracy via HyDE, BioBERT Re-ranking, and Targeted Prompt Engineering." *BioBERT Re-ranking, and Targeted Prompt Engineering*.
9. Xu, Shan, Zhaokun Yan, Chengxiao Dai, and Fan Wu. "MEGA-RAG: a retrieval-augmented generation framework with multi-evidence guided answer refinement for mitigating hallucinations of LLMs in public health." *Frontiers in Public Health* 13 (2025): 1635381.
10. Panchumarthi, Lovely Yeswanth, Sai Prasad Gudari, Atharva Negi, Praveen Raj Budime, and Harsit Upadhya. "RAG-BioQA Retrieval-Augmented Generation for Long-Form Biomedical Question Answering." *arXiv preprint arXiv:2510.01612* (2025).
11. Kim, Hyunjae, Jiwoong Sohn, Aidan Gilson, Nicholas Cochran-Caggiano, Serina Applebaum, Heeju Jin, Seihee Park et al. "Rethinking retrieval-augmented generation for medicine: A large-scale, systematic expert evaluation and practical insights." *arXiv preprint arXiv:2511.06738* (2025).
12. Stuhlmann, Linus, Michael Alexander Saxer, and Jonathan Fürst. "Efficient and Reproducible Biomedical Question Answering Using Retrieval Augmented Generation." In *2025 IEEE Swiss Conference on Data Science (SDS)*, pp. 154-157. IEEE, 2025.
13. Waikar, Shreyas, Amruta Gajanan Bhat, and Murali Ramanathan. "Retrieval Augmented Generation (RAG) for Evaluating Regulatory Compliance of Drug Information and Clinical Trial Protocols." *CPT: Pharmacometrics & Systems Pharmacology* 15, no. 3 (2026): e70201.
14. Ji, Yuelyu, MinGu Kwak, Hang Zhang, Xizhi Wu, Chenyu Li, and Yanshan Wang. "MedRAGChecker: Claim-Level Verification for Biomedical Retrieval-Augmented Generation." *arXiv preprint arXiv:2601.06519* (2026).
15. Sun, Mengzhou, Sendong Zhao, Jianyu Chen, Haochun Wang, and Bin Qin. "M-Eval: A Heterogeneity-Based Framework for Multi-evidence Validation in Medical RAG Systems." *arXiv preprint arXiv:2510.23995* (2025).

16. Amugongo, Lameck Mbangula, Pietro Mascheroni, Steven Brooks, Stefan Doering, and Jan Seidel. "Retrieval augmented generation for large language models in healthcare: A systematic review." *PLOS Digital Health* 4, no. 6 (2025): e0000877.
17. Advani, Nikhil, Amruta Gajanan Bhat, Sathy Balu-Iyer, and Murali Ramanathan. "Retrieval augmented generation (rag) for natural language querying of immunogenicity data for protein drugs." *The AAPS journal* 28, no. 2 (2026): 51.
18. Li, Changjin, Fengshi Jing, Huarun Li, Zheng Dong, Zhougzhi Xu, Huiru Zou, Qiting Wang et al. "Medical multi-recall embedding: Adaptive retrieval for diverse evidence in medical RAG systems." *Information Processing & Management* 63, no. 5 (2026): 104695.
19. Zhao, Xuejiao, Siyan Liu, Su-Yin Yang, and Chunyan Miao. "Medrag: Enhancing retrieval-augmented generation with knowledge graph-elicited reasoning for healthcare copilot." In *Proceedings of the ACM on Web Conference 2025*, pp. 4442-4457. 2025.
20. Ngo, Nghia Trung, Chien Van Nguyen, Franck Dernoncourt, and Thien Huu Nguyen. "Comprehensive and practical evaluation of retrieval-augmented generation systems for medical question answering." *arXiv preprint arXiv:2411.09213* (2024).
21. Abo El-Enen, Mohamed, Sally Saad, and Taymoor Nazmy. "A survey on retrieval-augmentation generation (RAG) models for healthcare applications." *Neural Computing and Applications* 37, no. 33 (2025): 28191- 28267.